



- پروژه در گروه‌های **چهار نفری** انجام می‌شود. نحوه گروه‌بندی در CW اطلاع‌رسانی می‌شود.
- همه موارد قابل تحویل برای پروژه را در یک فایل Zip با نام CA-Project-STDID1-STDID2-STDID3-STDID4.zip جمع‌آوری نموده و در سامانه CW بارگذاری نمایید (از هر گروه تنها یک نفر پروژه را بارگذاری نماید).
- هر کدام از پروژه‌ها را تنها ۳ گروه می‌توانند انتخاب کنند و اولویت با گروه‌هایی خواهد بود که پروژه مورد نظر را زودتر انتخاب نمایند.
- در صورت هرگونه سوال یا اشکال، آن را در تالار مربوط به پروژه موردنظر در صفحه درس در CW مطرح نمایید.
- توصیه می‌شود شروع پروژه را به روزهای آخر ماکول نفرمایید و در اسرع وقت کارهای اولیه پروژه را شروع نمایید.
- در صورت مشاهده تقلب کل نمرات تمرینات و پروژه صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- از ساعت ۱۸ روز ۹ تیرماه سامانه برای انتخاب گروه باز شده و تا ۲۴ ساعت فرصت انتخاب گروه را خواهید داشت.
- از ساعت ۱۸ روز ۱۰ تیرماه سامانه برای انتخاب پروژه باز خواهد شد و تا ۴۸ ساعت فرصت اخذ پروژه را خواهید داشت. در صورتیکه انتخاب گروه انجام شده باشد یکی از اعضای گروه پروژه مورد نظر را باید انتخاب کند (توجه داشته باشید از هر گروه تنها یک نفر پروژه مورد نظر را اخذ کند).
- هر گروه باید حداکثر تا ساعت ۲۳:۵۹ تاریخ ۱۲ تیرماه (جمعه) تیم پروژه و توصیف مختصر پروژه را در قالب یک فایل pdf تک صفحه‌ای در صفحه درس بارگذاری نماید.
- موعد انجام پروژه در روزهای آتی مشخص خواهد شد.
- پروژه‌ها به صورت مجازی به دستیاران آموزشی تحویل داده می‌شود. همه اعضای گروه باید برای این منظور حضور یافته و به همگی قسمت‌های پروژه تسلط داشته باشند.
- **پروژه‌های سخت‌تر و آسان‌تر از نظر نمره مقیاس می‌شوند.**
- گزارش پروژه باید در فرمت لاتک و در سامانه لاتک آنلاین دانشگاه نوشته شود. لذا یکی از نفرات پروژه باید قالب گزارش سمینار را از لینک ذیل انتخاب کرده و در این قالب گزارش تیم پروژه را ایجاد نماید. پروژه باید بین اعضای گروه به اشتراک گذاشته شود و تمامی اعضای گروه باید در نوشتار مشارکت نمایند. دقت شود تاریخچه مشارکت اعضای گروه، توسط دستیار آموزشی در سامانه قابل رویت خواهد بود.

آدرس ورود به سامانه لاتک دانشگاه: [Login Latex](#)
قالب گزارش پروژه: [Template Latex](#)

- گزارش پروژه یک و دو باید با آقای صداقت‌گو و آقای احمدی در سامانه به اشتراک گذاشته شود.
- گزارش پروژه‌های سه و چهار باید با آقای صداقت‌گو در سامانه به اشتراک گذاشته شود.
- گزارش پروژه پنج و شش باید با آقای صداقت‌گو و آقای میرزائی در سامانه به اشتراک گذاشته شود.
- گزارش پروژه هفتم و هشتم باید با آقای صداقت‌گو و آقای کوهی در سامانه به اشتراک گذاشته شود.
- ایمیل دستیاران آموزشی برای اشتراک گذاری پروژه‌ها:

Sedaghatgoo: ali.sedaghatgoo43@sharif.edu

Ahmadi: kasra.ahmadi04@sharif.edu

Mirzaei: amir.mirzaii22@sharif.edu

Koohi: amin.koohi85@sharif.edu

پروژه اول: افزودن حافظه نهان و مدل سازی تأخیر حافظه در پردازنده خط لوله ای

مقدمه

در معماری کامپیوتر، یکی از عوامل مهم محدودکننده کارایی پردازنده، فاصله ی سرعت میان پردازنده و حافظه ی اصلی است. حتی اگر پردازنده بتواند در هر چرخه یک دستور را اجرا کند، کند بودن دسترسی به حافظه باعث می شود پردازنده در بسیاری از چرخه ها منتظر آماده شدن داده یا دستور باقی بماند. برای کاهش این مشکل، در پردازنده های واقعی از سلسله مراتب حافظه^۱ و به خصوص حافظه نهان^۲ استفاده می شود.

در تمرین ها، پردازنده ای خط لوله ای^۳ در نرم افزار لاجسیم^۴ پیاده سازی شده است که به یک حافظه ی ساده متصل بوده و دسترسی به حافظه در آن بدون تأخیر در نظر گرفته شده است. در حالی که در سیستم های واقعی، دسترسی به حافظه ی اصلی معمولاً چندین چرخه زمان می برد و همین موضوع می تواند باعث توقف^۵ در خط لوله شود.

هدف این پروژه، اضافه کردن یک حافظه نهان ساده به پردازنده، مدل سازی تأخیر حافظه ی اصلی، و بررسی اثر حافظه نهان بر کاهش تعداد چرخه های توقف و بهبود عملکرد پردازنده است.

اهداف

۱. آشنایی با نقش سلسله مراتب حافظه در بهبود کارایی پردازنده
۲. مدل سازی تأخیر حافظه ی اصلی در یک پردازنده ی خط لوله ای
۳. طراحی و پیاده سازی حافظه نهان داده یا دستور در لاجسیم
۴. آشنایی با مفاهیم برخورد^۶، عدم برخورد^۷، جریمه ی عدم برخورد^۸ و توقف
۵. بررسی اثر الگوی دسترسی برنامه بر کارایی حافظه نهان
۶. مقایسه ی عملکرد پردازنده در حالت بدون حافظه نهان و با حافظه نهان

صورت پروژه

در این پروژه باید پردازنده ی خط لوله ای خود را که در تمرین های قبلی طراحی کرده اید، به گونه ای تغییر دهید که دسترسی به حافظه ی اصلی دیگر در یک چرخه انجام نشود. به عبارت دیگر، حافظه ی اصلی باید دارای تأخیر چند چرخه ای باشد. سپس باید یک حافظه نهان ساده میان پردازنده و حافظه ی اصلی اضافه کنید تا در صورت وجود داده یا دستور مورد نیاز در حافظه نهان، دسترسی سریع تر انجام شود.

در حالت اولیه، فرض کنید هر دسترسی مستقیم به حافظه ی اصلی حداقل چند چرخه زمان می برد. این تأخیر باید به صورت سخت افزاری و با استفاده از ثبات ها، شمارنده ها و سیگنال های کنترلی در لاجسیم پیاده سازی شود. در صورتی که پردازنده منتظر پاسخ حافظه باشد، خط لوله باید به شکل صحیح متوقف شود و پس از آماده شدن داده، اجرای برنامه ادامه پیدا کند. حافظه نهان طراحی شده باید رفتاری شفاف داشته باشد؛ یعنی برنامه نویس نباید نیاز داشته باشد برای استفاده از حافظه نهان کد برنامه را تغییر دهد. پردازنده باید به صورت عادی درخواست خواندن یا نوشتن را صادر کند و کنترل کننده ی حافظه نهان تصمیم بگیرد که آیا داده از حافظه نهان قابل دسترسی است یا باید از حافظه ی اصلی دریافت شود.

¹Memory Hierarchy

²Cache Memory

³Pipelined Processor

⁴Logisim

⁵Stall

⁶Hit

⁷Miss

⁸Miss Penalty

گام‌های پروژه

گام اول: مدل‌سازی تأخیر حافظه‌ی اصلی در ابتدا باید حافظه‌ی اصلی پردازنده‌ی خود را به گونه‌ای تغییر دهید که عملیات خواندن و نوشتن آن بلافاصله در همان چرخه انجام نشود. برای این کار باید یک مدار کنترل تأخیر طراحی کنید.

۱. برای هر درخواست خواندن یا نوشتن از حافظه، یک شمارنده فعال شود.
۲. تا زمانی که شمارنده به مقدار مشخص شده نرسیده است، حافظه سیگنال آماده‌بودن^۹ را فعال نکند.
۳. پس از پایان تأخیر، عملیات حافظه کامل شود و داده‌ی خوانده‌شده یا نتیجه‌ی نوشتن در اختیار پردازنده قرار گیرد.
۴. مقدار تأخیر حافظه باید قابل تنظیم باشد. برای مثال بتوانید حافظه را با تأخیرهای ۲، ۴ و ۸ چرخه آزمایش کنید.

در این گام، پردازنده باید بتواند در زمان انتظار برای حافظه، خط لوله را متوقف کند. توقف خط لوله باید به گونه‌ای باشد که مقادیر ثبات‌های میانی خط لوله خراب نشوند و پس از پایان تأخیر، اجرای برنامه از همان نقطه ادامه یابد.

گام دوم: طراحی کنترل توقف برای خط لوله در این گام باید سیگنال‌های لازم برای توقف خط لوله را طراحی و پیاده‌سازی کنید. اگر دستور فعلی نیاز به دسترسی به حافظه داشته باشد و حافظه هنوز آماده نباشد، مراحل مناسب خط لوله باید متوقف شوند.

موارد زیر باید رعایت شوند:

۱. در هنگام انتظار برای حافظه، مقدار شمارنده برنامه^{۱۰} نباید به اشتباه تغییر کند.
۲. ثبات‌های میانی خط لوله نباید داده‌ی نامعتبر دریافت کنند.
۳. پس از آماده‌شدن حافظه، اجرای دستور باید به صورت صحیح ادامه پیدا کند.
۴. اگر دستورهای بعدی هنوز اجازه‌ی اجرا ندارند، نباید وارد مرحله‌ای شوند که باعث تغییر نادرست وضعیت پردازنده شود.

در گزارش خود توضیح دهید که کدام مراحل خط لوله را متوقف کرده‌اید و چرا.

گام سوم: افزودن حافظه نهان ساده در این گام باید یک حافظه نهان کوچک میان پردازنده و حافظه‌ی اصلی اضافه کنید. این حافظه نهان باید حداقل برای دسترسی‌های داده استفاده شود. در صورت تمایل می‌توانید حافظه نهان را برای دستورهای نیز اضافه کنید، اما بخش اصلی پروژه مربوط به حافظه نهان داده است. حافظه نهان باید حداقل ویژگی‌های زیر را داشته باشد:

۱. نگاشت مستقیم^{۱۱}
۲. تعداد خطوط حافظه نهان^{۱۲} حداقل ۴ خط
۳. هر خط حافظه نهان شامل حداقل یک کلمه داده باشد
۴. برای هر خط حافظه نهان یک بیت معتبر^{۱۳} وجود داشته باشد
۵. برای تشخیص برخورد و عدم برخورد از برچسب^{۱۴} استفاده شود
۶. در صورت برخورد، داده بدون مراجعه به حافظه‌ی اصلی در اختیار پردازنده قرار گیرد

^۹Ready

^{۱۰}Program Counter

^{۱۱}Direct-Mapped Cache

^{۱۲}Cache Lines

^{۱۳}Valid Bit

^{۱۴}Tag

۷. در صورت عدم برخورد، پردازنده متوقف شود تا داده از حافظه‌ی اصلی دریافت و وارد حافظه نهان شود

در این گام، طراحی شما باید به گونه‌ای باشد که بتوان برخورد و عدم برخورد را از طریق سیگنال‌ها یا نمایشگرهای لاجسیم مشاهده کرد.

گام چهارم: سیاست نوشتن در حافظه نهان در این گام باید نحوه‌ی انجام عملیات نوشتن را مشخص کنید. برای ساده‌سازی، سیاست نوشتن پایه‌ی پروژه به صورت write-through در نظر گرفته می‌شود. یعنی در هر عملیات نوشتن، داده هم در حافظه نهان و هم در حافظه‌ی اصلی به‌روزرسانی شود. در پیاده‌سازی خود باید مشخص کنید:

۱. اگر آدرس مورد نظر در حافظه نهان وجود داشته باشد، داده‌ی حافظه نهان چگونه به‌روزرسانی می‌شود.

۲. اگر آدرس مورد نظر در حافظه نهان وجود نداشته باشد، آیا خط جدید وارد حافظه نهان می‌شود یا فقط حافظه‌ی اصلی تغییر می‌کند.

۳. پردازنده هنگام نوشتن در حافظه‌ی اصلی چند چرخه متوقف می‌شود.

برای سادگی، می‌توانید از سیاست write-through همراه با write-allocate یا no-write-allocate استفاده کنید، اما انتخاب خود را باید در گزارش توضیح دهید.

گام پنجم: شمارنده‌های آماری و ابزار ارزیابی برای ارزیابی عملکرد حافظه نهان، باید چند شمارنده در مدار خود اضافه کنید. این شمارنده‌ها باید حداقل موارد زیر را محاسبه کنند:

۱. تعداد کل دسترسی‌ها به حافظه

۲. تعداد برخوردها در حافظه نهان

۳. تعداد عدم برخوردها در حافظه نهان

۴. تعداد چرخه‌هایی که پردازنده به دلیل انتظار برای حافظه متوقف شده است

۵. تعداد کل چرخه‌های اجرای برنامه

با استفاده از این مقادیر، باید نرخ برخورد و نرخ عدم برخورد را محاسبه کنید:

نرخ برخورد = تعداد برخوردها / تعداد کل دسترسی‌ها به حافظه

نرخ عدم برخورد = تعداد عدم برخوردها / تعداد کل دسترسی‌ها به حافظه

همچنین باید عملکرد پردازنده را در دو حالت زیر مقایسه کنید:

۱. پردازنده با حافظه‌ی دارای تأخیر و بدون حافظه نهان

۲. پردازنده با حافظه‌ی دارای تأخیر و همراه با حافظه نهان

گام ششم: طراحی برنامه‌های آزمایشی برای بررسی رفتار حافظه نهان، باید چند برنامه‌ی کوچک بنویسید و روی پردازنده‌ی خود اجرا کنید. برنامه‌ها باید به گونه‌ای باشند که تفاوت میان برخورد و عدم برخورد و همچنین اثر الگوی دسترسی به حافظه قابل مشاهده باشد.

حداقل برنامه‌های زیر باید پیاده‌سازی و آزمایش شوند:

۱. برنامه‌ای با دسترسی ترتیبی به آرایه

۲. برنامه‌ای با دسترسی تکراری به چند آدرس محدود

۳. برنامه‌ای با دسترسی به آدرس‌هایی که باعث جایگزینی مکرر خطوط حافظه نهان شوند برای مثال می‌توانید از دو الگوی زیر استفاده کنید و تفاوت عملکرد آن‌ها را توضیح دهید:

```

1 #define N 16
2
3 int a[N][N];
4 int sum = 0;
5
6 // P1:
7 for (int i = 0; i < N; i++)
8   for (int j = 0; j < N; j++)
9     sum += a[i][j];
10
11 // P2:
12 for (int i = 0; i < N; i++)
13   for (int j = 0; j < N; j++)
14     sum += a[j][i];

```

در گزارش خود توضیح دهید که چرا این دو برنامه ممکن است نرخ برخورد متفاوتی داشته باشند و کدام یک برای حافظه نهان مناسب‌تر است. منظور از دسترسی سطری و ستونی، ترتیب دسترسی به خانه‌های آرایه در حافظه است؛ در برخی آرایش‌های حافظه، یکی از این الگوها می‌تواند باعث استفاده‌ی بهتر از هم‌جواری مکانی^{۱۵} شود.

گام هفتم: مقایسه و تحلیل نتایج در این گام باید آزمایش‌های خود را با چند مقدار مختلف برای تأخیر حافظه اجرا کنید. حداقل سه حالت زیر را بررسی کنید:

۱. تأخیر حافظه برابر با ۲ چرخه

۲. تأخیر حافظه برابر با ۴ چرخه

۳. تأخیر حافظه برابر با ۸ چرخه

برای هر حالت، نتایج زیر را گزارش کنید:

۱. تعداد کل چرخه‌های اجرا بدون حافظه نهان

۲. تعداد کل چرخه‌های اجرا با حافظه نهان

۳. تعداد برخوردها و عدم برخوردها

۴. نرخ برخورد و نرخ عدم برخورد

۵. تعداد چرخه‌های توقف

۶. میزان بهبود عملکرد نسبت به حالت بدون حافظه نهان

نتایج خود را در قالب جدول و نمودار ارائه دهید و توضیح دهید که افزایش تأخیر حافظه چه اثری بر اهمیت حافظه نهان دارد.

¹⁵Spatial Locality

قسمت امتیازی

برای دریافت امتیاز اضافه، می‌توانید یکی از موارد زیر را انجام دهید:

۱. افزودن حافظه نهان جداگانه برای دستورها و داده‌ها
۲. پیاده‌سازی حافظه نهان مجموعه‌ای دوسویه^{۱۶}
۳. استفاده از سیاست write-back به همراه بیت کثیف^{۱۷}
۴. اضافه کردن بلوک‌های چندکلمه‌ای به حافظه نهان و بررسی اثر اندازه‌ی بلوک
۵. محاسبه‌ی میانگین چرخه به‌ازای هر دستور^{۱۸} در حالت بدون حافظه نهان و با حافظه نهان و تحلیل اختلاف آن‌ها

گزارش و ارزیابی

در گزارش پروژه باید موارد زیر را توضیح دهید:

۱. نحوه‌ی مدل‌سازی تأخیر حافظه‌ی اصلی
۲. نحوه‌ی توقف خط لوله هنگام آماده‌نبودن حافظه
۳. ساختار حافظه نهان شامل شاخص^{۱۹}، برجسب، بیت معتبر و داده
۴. نحوه‌ی تشخیص برخورد و عدم برخورد
۵. سیاست مورد استفاده برای عملیات نوشتن
۶. شمارنده‌های آماری طراحی شده و نحوه‌ی محاسبه‌ی آن‌ها
۷. برنامه‌های آزمایشی استفاده‌شده و دلیل انتخاب آن‌ها
۸. مقایسه‌ی عملکرد پردازنده در حالت بدون حافظه نهان و با حافظه نهان
۹. تحلیل چالش‌های پیاده‌سازی و خطاهایی که در طول پروژه با آن‌ها مواجه شده‌اید

نحوه‌ی تحویل

تمامی موارد زیر باید در یک پوشه ذخیره و تحویل داده شوند:

۱. فایل کامل مدار لاجیسیم
۲. فایل یا فایل‌های برنامه‌های آزمایشی اجراشده روی پردازنده
۳. توضیحات نحوه‌ی اجرای مدار و تنظیم مقدار تأخیر حافظه
۴. جدول نتایج مربوط به برخورد، عدم برخورد، توقف و تعداد چرخه‌های اجرا
۵. نمودارهای مقایسه‌ای عملکرد در حالت بدون حافظه نهان و با حافظه نهان
۶. گزارش کامل شامل توضیح طراحی، پیاده‌سازی، نتایج، تحلیل نتایج و چالش‌های انجام پروژه

¹⁶Two-Way Set Associative Cache

¹⁷Dirty Bit

¹⁸Cycles Per Instruction

¹⁹Index

پروژه دوم: پیاده‌سازی و تحلیل سیاست‌های جایگزینی حافظه نهان در شبیه‌ساز ChampSim

مقدمه

با افزایش فاصله‌ی سرعت میان پردازنده و حافظه‌ی اصلی، حافظه نهان^{۲۰} به یکی از اجزای مهم در بهبود کارایی سامانه‌های کامپیوتری تبدیل شده است. حافظه نهان با نگهداری داده‌ها و دستورهای که احتمال استفاده‌ی مجدد از آن‌ها بیشتر است، می‌تواند زمان متوسط دسترسی به حافظه را کاهش دهد. با این حال، ظرفیت حافظه نهان محدود است و در بسیاری از مواقع، هنگام ورود یک بلوک جدید، لازم است یکی از بلوک‌های موجود از حافظه نهان خارج شود. تصمیم‌گیری درباره‌ی انتخاب این بلوک، توسط سیاست جایگزینی^{۲۱} انجام می‌شود.

در تمرین، دانشجویان با اجرای شبیه‌ساز ChampSim، تغییر پیکربندی حافظه نهان سطح آخر^{۲۲} و بررسی اثر سیاست‌های ساده‌ی جایگزینی مانند LRU، FIFO، MRU و Random آشنا شده‌اند. در این پروژه، هدف این است که چند سیاست جایگزینی پیشرفته‌تر در همین شبیه‌ساز پیاده‌سازی و تحلیل شود. تمرکز اصلی پروژه بر این است که مشخص شود سیاست‌های پیچیده‌تر، در ردپاها^{۲۳} و الگوهای دسترسی مختلف، تا چه اندازه می‌توانند نرخ عدم برخورد^{۲۴} را کاهش دهند و کارایی پردازنده را بهبود دهند.

اهمیت این موضوع در حافظه نهان سطح آخر بیشتر است؛ زیرا عدم برخورد^{۲۵} در این سطح معمولاً هزینه‌ی بیشتری نسبت به سطوح بالاتر حافظه نهان دارد و می‌تواند باعث افزایش زمان انتظار پردازنده و کاهش تعداد دستورهای اجراشده در هر چرخه^{۲۶} شود. بنابراین یک سیاست جایگزینی مناسب باید بتواند بلوک‌هایی را در حافظه نهان نگه دارد که احتمال استفاده‌ی مجدد از آن‌ها بیشتر است و بلوک‌هایی را حذف کند که در آینده‌ی نزدیک کمتر مورد استفاده قرار می‌گیرند.

اهداف

۱. پیاده‌سازی چند سیاست جایگزینی پیشرفته برای حافظه نهان سطح آخر در شبیه‌ساز ChampSim
۲. مقایسه‌ی سیاست‌های ساده و پیشرفته از نظر نرخ برخورد^{۲۷}، نرخ عدم برخورد و کارایی کلی پردازنده
۳. تحلیل اثر الگوی دسترسی برنامه‌ها بر عملکرد سیاست‌های جایگزینی
۴. بررسی رابطه‌ی میان کاهش عدم برخورد و افزایش تعداد دستورهای اجراشده در هر چرخه
۵. مقایسه‌ی پیچیدگی پیاده‌سازی سیاست‌ها از نظر اطلاعات کمکی، شمارنده‌ها و منطق انتخاب بلوک قربانی^{۲۸}

صورت پروژه

در این پروژه باید چند سیاست جایگزینی حافظه نهان را در شبیه‌ساز ChampSim و برای حافظه نهان سطح آخر پیاده‌سازی و ارزیابی کنید. فرض می‌شود که دانشجویان با نحوه‌ی اجرای ChampSim، فایل پیکربندی `champsim_config.json`، اجرای فایل‌های ردپا و استخراج آمار مربوط به LLC آشنا هستند. بنابراین در این پروژه نیازی به توضیح مقدمات کار با شبیه‌ساز نیست و تمرکز اصلی باید روی پیاده‌سازی سیاست‌ها، طراحی آزمایش‌های منصفانه و تحلیل نتایج باشد. سیاست‌های پایه که باید به عنوان مبنای مقایسه در نظر گرفته شوند عبارت‌اند از:

²⁰Cache Memory

²¹Replacement Policy

²²Last-Level Cache

²³Traces

²⁴Miss Rate

²⁵Miss

²⁶Instructions Per Cycle

²⁷Hit Rate

²⁸Victim Block

- LRU
- FIFO
- MRU
- Random

علاوه بر سیاست‌های پایه، هر گروه باید حداقل چهار سیاست از فهرست زیر را پیاده‌سازی و بررسی کند:

- SRRIP
- BRRIP
- BIP
- DIP
- DRRIP
- Tree-PLRU
- LFU
- SHiP

اگر برخی از سیاست‌های پایه یا پیشرفته از قبل در نسخه‌ی مورد استفاده‌ی شما از ChampSim وجود داشته باشند، می‌توانید از همان پیاده‌سازی استفاده کنید؛ اما باید در گزارش مشخص کنید کدام سیاست را خودتان پیاده‌سازی کرده‌اید و کدام سیاست از پیاده‌سازی موجود استفاده شده است. برای سیاست‌هایی که خودتان اضافه می‌کنید، لازم است منطق انتخاب بلوک قربانی، نحوه‌ی به‌روزرسانی اطلاعات کمکی و تغییرات انجام‌شده در کد را توضیح دهید.

گام‌های پروژه

گام اول: در این گام باید پیکربندی پایه‌ای را که برای همه‌ی آزمایش‌ها استفاده می‌شود مشخص کنید. هدف این است که مقایسه‌ی سیاست‌ها منصفانه باشد و تفاوت نتایج تا حد امکان فقط ناشی از سیاست جایگزینی باشد، نه تغییر هم‌زمان چند پارامتر دیگر. در این بخش باید موارد زیر را ثابت نگه دارید:

۱. اندازه‌ی کلی حافظه نهان سطح آخر
۲. اندازه‌ی بلوک حافظه نهان^{۲۹}
۳. میزان هم‌انجمنی^{۳۰}، مگر در آزمایشی که به صورت مشخص اثر هم‌انجمنی را بررسی می‌کند
۴. تعداد دستورهای گرم‌سازی^{۳۱}
۵. تعداد دستورهای مرحله‌ی اصلی شبیه‌سازی^{۳۲}

²⁹Cache Block Size

³⁰Associativity

³¹Warmup Instructions

³²Simulation Instructions

۶. سایر پارامترهای پردازنده و حافظه، به جز سیاست جایگزینی

سیاست LRU را به عنوان مبنای اصلی مقایسه در نظر بگیرید و عملکرد سایر سیاست‌ها را نسبت به آن گزارش کنید.
گام دوم: در این گام باید سیاست‌های انتخاب‌شده را از نظر منطق تصمیم‌گیری مطالعه و دسته‌بندی کنید. لازم نیست توضیحات عمومی و مقدماتی درباره‌ی ChampSim نوشته شود؛ بلکه باید مشخص کنید هر سیاست با چه ایده‌ای بلوک قربانی را انتخاب می‌کند.

برای هر سیاست، موارد زیر را در گزارش توضیح دهید:

۱. اطلاعات کمکی مورد نیاز سیاست برای هر خط حافظه نهان

۲. نحوه‌ی انتخاب بلوک قربانی هنگام وقوع عدم برخورد

۳. نحوه‌ی به‌روزرسانی اطلاعات سیاست پس از برخورد

۴. نحوه‌ی به‌روزرسانی اطلاعات سیاست پس از جایگزینی

۵. شرایطی که انتظار دارید سیاست در آن عملکرد مناسب یا نامناسب داشته باشد

برای مثال، در سیاست‌های مبتنی بر پیش‌بینی فاصله‌ی مراجعه‌ی مجدد^{۳۳} مانند SRRIP، باید مشخص کنید مقدار پیش‌بینی مراجعه‌ی مجدد^{۳۴} چگونه نگهداری و به‌روزرسانی می‌شود. همچنین در سیاست‌های سازگار شونده^{۳۵} مانند DIP یا DRRIP باید توضیح دهید سیاست چگونه میان دو رفتار مختلف انتخاب می‌کند.

گام سوم: در این گام باید سیاست‌های انتخاب‌شده را در ChampSim پیاده‌سازی کنید. برای هر سیاست باید فایل‌های لازم را ایجاد یا تکمیل کنید و مطمئن شوید که سیاست جدید از طریق پیکربندی شبیه‌ساز قابل انتخاب و اجرا است. در پیاده‌سازی هر سیاست باید حداقل سه بخش اصلی مشخص باشد:

۱. مقداردهی اولیه‌ی داده‌های کمکی سیاست

۲. انتخاب بلوک قربانی در زمان عدم برخورد

۳. به‌روزرسانی وضعیت سیاست پس از برخورد، عدم برخورد و جایگزینی

در پیاده‌سازی خود باید دقت کنید که سیاست جایگزینی فقط بر اساس اطلاعات مجاز همان سیاست تصمیم بگیرد. برای مثال، اگر سیاست FIFO را بررسی می‌کنید، نباید از زمان آخرین استفاده‌ی بلوک در تصمیم‌گیری استفاده شود. همچنین اگر سیاست Random را به عنوان مبنا استفاده می‌کنید، انتخاب قربانی نباید در عمل به یک ترتیب ثابت و قابل پیش‌بینی تبدیل شود.

گام چهارم: در این گام باید چند ردپا با رفتارهای متفاوت انتخاب کنید. برخلاف تمرینی که فقط روی یک ردپای مشخص انجام شده بود، در این پروژه باید عملکرد سیاست‌ها در چند نوع الگوی دسترسی بررسی شود. حداقل چهار ردپا باید انتخاب و اجرا شود.

بهتر است ردپاها به گونه‌ای انتخاب شوند که رفتارهای زیر در آزمایش‌ها دیده شود:

۱. ردپایی با هم‌جواری زمانی^{۳۶} بالا

۲. ردپایی با هم‌جواری مکانی^{۳۷} بالا

۳. ردپایی با رفتار جریانی^{۳۸} و عدم برخوردهای زیاد

³³Re-Reference Interval Prediction

³⁴Re-Reference Prediction Value

³⁵Adaptive Policies

³⁶Temporal Locality

³⁷Spatial Locality

³⁸Streaming Behavior

۴. ردپایی که نسبت به تغییر سیاست جایگزینی حساسیت زیادی دارد

برای هر ردپا، همه‌ی سیاست‌ها باید با تعداد دستورهای گرم‌سازی و تعداد دستورهای شیشه‌سازی یکسان اجرا شوند. اگر زمان اجرای ردپاها بسیار متفاوت است، می‌توانید تعداد دستورها را طوری تنظیم کنید که اجرای هر آزمایش در زمان قابل قبول انجام شود؛ اما این تصمیم باید در گزارش توضیح داده شود.

گام پنجم: در این گام باید شیشه‌سازی را برای همه‌ی ترکیب‌های سیاست جایگزینی و ردپاهای انتخاب‌شده اجرا کنید. خروجی هر اجرا باید ذخیره شود تا بعداً بتوان نتایج را استخراج، بررسی و بازتولید کرد. برای هر ترکیب از ردپا و سیاست جایگزینی، حداقل معیارهای زیر را استخراج کنید:

۱. تعداد کل دسترسی‌ها به حافظه نهان سطح آخر

۲. تعداد برخوردها در حافظه نهان سطح آخر

۳. تعداد عدم برخوردها در حافظه نهان سطح آخر

۴. نرخ برخورد و نرخ عدم برخورد

۵. تعداد عدم برخورد به ازای هر هزار دستور^{۳۹}

۶. تعداد دستورهای اجراشده در هر چرخه

۷. میزان بهبود یا افت عملکرد نسبت به LRU

برای محاسبه‌ی بهبود عملکرد نسبت به LRU می‌توانید از رابطه‌ی زیر استفاده کنید:

$$Speedup = IPC(policy) / IPC(LRU)$$

نتایج باید در قالب جدول ذخیره شوند. همچنین برای معیارهای مهم مانند نرخ عدم برخورد، IPC و بهبود عملکرد نسبت به LRU نمودار مناسب رسم کنید.

گام ششم: در این گام باید نتایج به دست آمده را تحلیل کنید. صرفاً ارائه‌ی جدول و نمودار کافی نیست. باید توضیح دهید که چرا یک سیاست در یک ردپا بهتر و در ردپای دیگر ضعیف‌تر عمل کرده است. در تحلیل خود به پرسش‌های زیر پاسخ دهید:

۱. کدام سیاست در میانگین همه‌ی ردپاها بهترین عملکرد را داشته است؟

۲. آیا سیاستی وجود دارد که برای همه‌ی ردپاها بهترین باشد؟

۳. در کدام ردپاها سیاست‌های پیشرفته نسبت به LRU بهتر عمل کرده‌اند؟

۴. در کدام ردپاها سیاست‌های ساده مانند LRU یا FIFO کافی بوده‌اند؟

۵. آیا کاهش نرخ عدم برخورد همیشه باعث افزایش IPC شده است؟

۶. رفتار سیاست‌های سازگار شونده را چگونه تفسیر می‌کنید؟

حداقل نمودارهای زیر باید در گزارش ارائه شوند:

۱. نمودار نرخ عدم برخورد برای همه‌ی سیاست‌ها و ردپاها

۲. نمودار IPC برای همه‌ی سیاست‌ها و ردپاها

³⁹Misses Per Kilo Instructions

۳. نمودار بهبود عملکرد نسبت به LRU

گام هفتم: در این گام باید سیاست‌ها را از نظر پیچیدگی پیاده‌سازی و هزینه‌ی تقریبی سخت‌افزاری مقایسه کنید. منظور از هزینه، مواردی مانند تعداد بیت‌های کمکی، نیاز به شمارنده‌های سراسری، نیاز به منطق انتخاب پیچیده، به‌روزرسانی‌های احتمالی و سربار ذخیره‌سازی اطلاعات سیاست است. برای هر سیاست، موارد زیر را بررسی کنید:

۱. تعداد بیت‌های کمکی لازم برای هر خط حافظه نهان

۲. نیاز یا عدم نیاز به شمارنده‌های سراسری

۳. پیچیدگی انتخاب بلوک قربانی

۴. پیچیدگی به‌روزرسانی وضعیت سیاست پس از برخورد و عدم برخورد

۵. مناسب بودن یا نبودن سیاست برای پیاده‌سازی سخت‌افزاری

در پایان این بخش توضیح دهید که اگر صرفاً کارایی مهم باشد، کدام سیاست را انتخاب می‌کنید و اگر سادگی سخت‌افزاری نیز مهم باشد، انتخاب شما چه تغییری می‌کند.

قسمت امتیازی

برای دریافت امتیاز اضافه، می‌توانید یکی از موارد زیر را انجام دهید:

۱. پیاده‌سازی سیاست SHiP و مقایسه‌ی آن با SRRIP و DRRIP

۲. پیاده‌سازی سیاست Hawkeye یا یک نسخه‌ی ساده‌شده از آن

۳. طراحی یک سیاست ترکیبی جدید و مقایسه‌ی آن با سیاست‌های پیاده‌سازی‌شده

۴. بررسی اثر اندازه‌ی حافظه نهان سطح آخر بر عملکرد سیاست‌ها

۵. بررسی اثر میزان هم‌انجمنی بر عملکرد سیاست‌ها، در حالی که اندازه‌ی کلی حافظه نهان ثابت بماند

۶. نوشتن اسکریپت برای اجرای خودکار همه‌ی آزمایش‌ها و تولید خودکار جدول‌ها و نمودارها

۷. تحلیل اثر سیاست جایگزینی بر پهنای‌باند مصرفی حافظه‌ی اصلی

نحوه‌ی تحویل گزارش

تمامی موارد ذیل باید در یک پوشه ذخیره و تحویل داده شوند:

۱. کد کامل ChampSim شامل سیاست‌های جایگزینی پیاده‌سازی‌شده

۲. فایل‌های پیکربندی استفاده‌شده برای آزمایش‌ها

۳. اسکریپت‌های اجرای آزمایش‌ها و استخراج نتایج

۴. فایل‌های خروجی شبیه‌سازی برای هر ردپا و هر سیاست

۵. جدول‌های نهایی و نمودارهای مقایسه‌ای

۶. تحلیل‌های متنی و نموداری حاصل از اجرای برنامه‌ها

۷. گزارش کامل از کارهای انجام‌شده شامل توضیح مسئله، سیاست‌های پیاده‌سازی‌شده، تغییرات انجام‌شده در کد، تنظیمات آزمایش، نتایج، تحلیل نتایج، مقایسه‌ی پیچیدگی پیاده‌سازی و چالش‌های انجام پروژه

پروژه سوم: تحلیل گلوگاه‌های سخت‌افزاری استنتاج مدل‌های زبانی بزرگ بر روی یک کارت گرافیک

مقدمه

مدل‌های زبانی بزرگ^{۴۰} امروزه یکی از مهم‌ترین کاربردهای پردازش موازی و معماری پردازنده‌های گرافیکی محسوب می‌شوند. اگرچه کارت‌های گرافیکی مدرن توان پردازشی بسیار بالایی دارند، اما عملکرد استنتاج مدل‌های زبانی بزرگ تنها به قدرت پردازشی آن‌ها وابسته نیست. در بسیاری از موارد، گلوگاه عملکرد سیستم می‌تواند ناشی از محدودیت پهنای باند حافظه، ظرفیت حافظه گرافیکی، انتقال داده از طریق گذرگاه PCIe، پردازنده مرکزی، حافظه اصلی و یا نحوه مدیریت حافظه توسط زمان اجرای CUDA باشد.

یکی از معروف‌ترین چارچوب‌های اجرای مدل‌های زبانی بزرگ، vLLM است که از قابلیت CPU Memory Offloading پشتیبانی می‌کند. در این روش بخشی از پارامترهای مدل به جای حافظه‌ی کارت گرافیکی، در حافظه‌ی اصلی سیستم ذخیره شده و در زمان نیاز از طریق گذرگاه PCIe به کارت گرافیکی منتقل می‌شوند. این قابلیت امکان اجرای مدل‌های بزرگ‌تر از ظرفیت حافظه‌ی کارت گرافیکی را فراهم می‌کند، اما ممکن است باعث ایجاد گلوگاه‌های سخت‌افزاری جدید شود. در این پروژه دانشجویان با استفاده از ابزارهای تحلیل عملکرد نظیر nvidia-smi، Nsight Systems، eBPF و ابزارهای پروفایلینگ CUDA عملکرد سیستم را بررسی کرده و گلوگاه اصلی اجرای مدل‌های زبانی بزرگ را از دیدگاه معماری کامپیوتر تحلیل خواهند کرد.

توجه:

برای انجام این پروژه به یک سیستم عامل لینوکس نصب‌شده به صورت Bare Metal و یک کارت گرافیک NVIDIA دارای پشتیبانی از CUDA نیاز است.

اهداف

۱. آشنایی با معماری اجرای مدل‌های زبانی بزرگ بر روی کارت‌های گرافیکی
۲. شناخت مسیر انتقال داده بین پردازنده، حافظه اصلی، گذرگاه PCIe و حافظه کارت گرافیکی
۳. تحلیل گلوگاه‌های سخت‌افزاری شامل توان پردازشی، حافظه، پهنای باند و انتقال داده
۴. آشنایی با ابزارهای پروفایلینگ و تحلیل عملکرد سیستم

صورت پروژه

هدف این پروژه تحلیل عملکرد اجرای مدل‌های زبانی بزرگ بر روی یک سیستم تک‌کارت‌گرافیکی است. در این پروژه دانشجویان باید با اجرای یک مدل زبانی بزرگ در دو حالت مختلف شامل اجرای عادی و اجرای همراه با CPU Memory Offloading رفتار سخت‌افزاری سیستم را تحلیل کنند. در طول اجرای برنامه، اطلاعات مربوط به فراخوانی‌های CUDA و همچنین میزان استفاده از پردازنده، کارت گرافیکی، حافظه، انتقال داده از طریق PCIe و اجرای کرنل‌های CUDA جمع‌آوری شده و در نهایت مشخص شود که گلوگاه اصلی سیستم در هر سناریو مربوط به کدام بخش از معماری کامپیوتر است.

گام‌های پروژه

گام اول:

در ابتدا درایورهای NVIDIA و CUDA را نصب کرده و یک مثال ساده از CUDA (مانند vectorAdd) را اجرا نمایید تا از صحت نصب محیط اطمینان حاصل شود.

⁴⁰Large Language Models (LLMs)

گام دوم:

به کمک یکی از ابزارهای تحلیل عملکرد مانند eBPF، Nsight Systems یا CUDA Profiling API فراخوانی‌های مهم از جمله `cudaMalloc`، `cudaMemcpy`، `cudaLaunchKernel` را ثبت کرده و زمان اجرای هر کدام را اندازه‌گیری نمایید.

گام سوم:

یک مدل زبانی مناسب (به گونه‌ای که حدود 70% از حافظه کارت گرافیکی را اشغال کند) در vLLM اجرا کرده و با استفاده از GuideLLM بارکاری مناسبی بر روی سیستم اعمال نمایید. در این مرحله معیارهای زیر اندازه‌گیری شوند:

- Time to First Token (TTFT)
- Throughput
- GPU Utilization
- GPU Memory Usage
- CPU Utilization
- Main Memory Usage
- CUDA Kernel Execution Time

گام چهارم:

با استفاده از گزینه `--cpu-offload-gb` بخشی از مدل را به حافظه اصلی منتقل کرده و آزمایش‌های مرحله قبل را تکرار نمایید. علاوه بر معیارهای قبلی، موارد زیر نیز اندازه‌گیری شوند:

- میزان انتقال داده بین CPU و GPU
- پهنای باند مورد استفاده در گذرگاه PCIe
- تغییرات زمان اجرای کرنل‌های CUDA
- زمان انتظار کارت گرافیکی برای دریافت داده

گام پنجم:

نتایج آزمایش‌های خود را تحلیل کرده و مشخص نمایید که در هر یک از سناریوهای اجرا، گلوگاه اصلی سیستم مربوط به کدام بخش از معماری کامپیوتر است. به عنوان مثال مشخص نمایید که آیا سیستم در هر آزمایش:

- Compute-bound
- GPU Memory-bound
- PCIe Bandwidth-bound
- CPU-bound
- Main Memory-bound

بوده است.

همچنین علت این رفتار را با استناد به نمودارها و داده‌های اندازه‌گیری شده توضیح دهید.

قسمت امتیازی

در این قسمت دانشجویان می‌توانند یکی از موارد زیر را انجام دهند:

۱. بررسی تاثیر Pinned Memory بر عملکرد انتقال داده بین CPU و GPU و مقایسه آن با حافظه معمولی.
۲. اندازه‌گیری میزان زمان صرف‌شده در فضای کاربر، فضای هسته و اجرای کرنل‌های CUDA و تحلیل سهم هر کدام در زمان اجرای کل برنامه.
۳. بررسی تاثیر تغییر اندازه Batch Size بر میزان استفاده از GPU، پهنای باند حافظه و عملکرد سیستم.

لینک‌های مفید

- CUDA Toolkit Documentation
<https://docs.nvidia.com/cuda/>
- CUDA C++ Programming Guide
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- CUDA Samples (VectorAdd, MatrixMul, etc.)
<https://github.com/NVIDIA/cuda-samples>
- NVIDIA Nsight Systems
<https://developer.nvidia.com/nsight-systems>
- NVIDIA Nsight Compute
<https://developer.nvidia.com/nsight-compute>
- NVIDIA System Management Interface (nvidia-smi)
<https://developer.nvidia.com/system-management-interface>
- NVIDIA CUDA Profiling Tools Interface (CUPTI)
<https://developer.nvidia.com/cupti>
- vLLM Documentation
<https://docs.vllm.ai/en/stable/>
- vLLM GitHub Repository
<https://github.com/vllm-project/vllm>
- Hugging Face Transformers
<https://huggingface.co/docs/transformers>
- GuideLLM Benchmark Tool
<https://github.com/vllm-project/guidellm>
- eBPF Official Website (Optional Monitoring Tool)
<https://ebpf.io/>
- NVIDIA Container Toolkit
<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

- Installing NVIDIA Drivers on Ubuntu 24.04
<https://linuxconfig.org/how-to-install-nvidia-drivers-on-ubuntu-24-04>
- FlashAttention Repository
<https://github.com/Dao-AILab/flash-attention>
- LLM Inference Optimization (NVIDIA Developer Blog)
<https://developer.nvidia.com/blog/>

پروژه چهارم: تحلیل تاثیر سلسله مراتب حافظه و ذخیره سازی بر استنتاج مدل های زبانی بزرگ

مقدمه

یکی از مهم ترین چالش های اجرای مدل های زبانی بزرگ بر روی سخت افزارهای امروزی، محدودیت ظرفیت حافظه ی کارت های گرافیکی است. اگرچه پردازنده های گرافیکی مدرن توان پردازشی بسیار بالایی دارند، اما ظرفیت حافظه ی آن ها در مقایسه با اندازه ی مدل های زبانی بزرگ و همچنین حافظه ی نهان (KV Cache) محدود است. به همین دلیل، در بسیاری از کاربردهای عملی، بخشی از داده ها باید از حافظه ی کارت گرافیکی به حافظه ی اصلی سیستم و در نهایت به حافظه ی جانبی منتقل شوند.

این فرآیند در واقع حرکت داده در سلسله مراتب حافظه ی سیستم است؛ از حافظه ی کارت گرافیکی (HBM/GDDR) به حافظه ی اصلی (DRAM) و در نهایت به حافظه ی جانبی (NVMe SSD). هر سطح از این سلسله مراتب دارای ظرفیت، پهنای باند و تاخیر متفاوتی است که مستقیماً بر عملکرد استنتاج مدل های زبانی بزرگ تاثیر می گذارد.

امروزه بسیاری از موتورهای استنتاج مدل های زبانی بزرگ از جمله vLLM برای غلبه بر محدودیت حافظه ی کارت گرافیکی، از روش هایی مانند CPU Memory Offloading و سامانه هایی نظیر LMCache استفاده می کنند. در این روش ها بخشی از KV Cache یا پارامترهای مدل در حافظه ی اصلی و یا حافظه ی جانبی نگهداری شده و هنگام نیاز از طریق گذرگاه PCIe به کارت گرافیکی منتقل می شوند. در نتیجه، گلوگاه سیستم دیگر لزوماً توان پردازشی کارت گرافیکی نیست، بلکه ممکن است پهنای باند حافظه، تاخیر انتقال داده یا سرعت ذخیره سازی به عامل محدودکننده ی عملکرد تبدیل شود.

هدف این پروژه بررسی تاثیر سطوح مختلف سلسله مراتب حافظه بر عملکرد استنتاج مدل های زبانی بزرگ است. دانشجویان با اجرای یک مدل زبانی بر روی یک سیستم تک کارت گرافیکی و اعمال سناریوهای مختلف KV Cache Offloading عملکرد سیستم را اندازه گیری کرده و مشخص خواهند کرد که در هر سناریو، گلوگاه اصلی سیستم مربوط به کدام بخش از معماری کامپیوتر است.

توجه:

برای انجام این پروژه به یک سیستم دارای سیستم عامل لینوکس به صورت Bare Metal و یک کارت گرافیک NVIDIA با پشتیبانی از CUDA نیاز است. همچنین پیشنهاد می شود از Ubuntu 24.04 به همراه آخرین نسخه ی درایورهای NVIDIA استفاده شود.

اهداف

- آشنایی با سلسله مراتب حافظه در سیستم های مجهز به GPU و نقش هر سطح از حافظه در اجرای بارهای کاری هوش مصنوعی.
- شناخت تفاوت ظرفیت، پهنای باند و تاخیر حافظه های HBM/GDDR، DRAM و NVMe SSD و تاثیر آن ها بر عملکرد سیستم.
- آشنایی با مفهوم KV Cache و نقش آن در استنتاج مدل های زبانی بزرگ.
- تحلیل تاثیر انتقال داده بین GPU، حافظه ی اصلی و حافظه ی جانبی بر معیارهای عملکردی مانند Time to First Token (TTFT) و Throughput.
- تحلیل گلوگاه های معماری شامل توان پردازشی GPU، ظرفیت حافظه، پهنای باند حافظه، گذرگاه PCIe و سامانه ی ذخیره سازی.
- آشنایی با ابزارهای تحلیل عملکرد و پروفایلینگ مانند nvidia-smi، iostat، fio و perf به منظور تحلیل رفتار معماری سیستم.
- توانایی تحلیل تجربی و ارائه ی استدلال معماری برای تشخیص گلوگاه اصلی اجرای مدل های زبانی بزرگ در سناریوهای مختلف.

صورت پروژه

در این پروژه دانشجویان باید بررسی کنند که زمانی که ظرفیت حافظه‌ی کارت گرافیکی برای نگهداری کامل KV Cache کافی نباشد، انتقال داده به سطوح پایین‌تر سلسله‌مراتب حافظه چه تاثیری بر عملکرد استنتاج مدل‌های زبانی بزرگ دارد. به طور خاص، هدف پروژه بررسی این موضوع است که آیا عملکرد سیستم توسط توان پردازشی کارت گرافیکی محدود می‌شود یا محدودیت‌هایی مانند ظرفیت حافظه، پهنای باند حافظه، گذرگاه PCIe و سامانه‌ی ذخیره‌سازی به گلوگاه اصلی تبدیل می‌شوند. برای این منظور، دانشجویان باید یک مدل زبانی بزرگ را با استفاده از vLLM و LMCache بر روی یک سیستم تک‌کارت‌گرافیکی اجرا کرده و عملکرد آن را در چندین سناریوی مختلف مقایسه کنند. در هر مرحله، نتایج به‌دست‌آمده باید تحلیل شده و مشخص شود که کدام بخش از معماری کامپیوتر عامل اصلی کاهش عملکرد سیستم است.

گام‌های پروژه

گام اول: راه‌اندازی محیط اجرا

در ابتدا درایورهای NVIDIA و CUDA را بر روی سیستم نصب نمایید. سپس NVIDIA Container Toolkit را نصب کرده و محیط اجرای vLLM و LMCache را راه‌اندازی کنید. پیشنهاد می‌شود از Ubuntu 24.04 و نسخه‌های رسمی Docker استفاده شود.

گام دوم: اجرای پایه (GPU-Only)

یک مدل زبانی انتخاب کنید که تقریباً 70% حافظه‌ی کارت گرافیکی را اشغال کند (برای مثال یکی از مدل‌های Qwen یا Llama). ابتدا مدل را بدون هیچ‌گونه KV Cache Offloading اجرا کنید؛ به گونه‌ای که تمام داده‌ها در حافظه‌ی کارت گرافیکی قرار داشته باشند. در این مرحله معیارهای زیر را اندازه‌گیری و ثبت نمایید:

• Time to First Token (TTFT)

• Throughput

• میزان استفاده از GPU

• میزان استفاده از حافظه‌ی GPU

• میزان استفاده از CPU

این مرحله به عنوان خط مبنا (Baseline) برای تمامی آزمایش‌های بعدی در نظر گرفته می‌شود.

گام سوم: انتقال KV Cache به حافظه‌ی اصلی

با استفاده از قابلیت CPU Memory Offloading بخشی از KV Cache را به حافظه‌ی اصلی (DRAM) منتقل کنید. سپس همان بارکاری مرحله‌ی قبل را اجرا کرده و معیارهای عملکرد را دوباره اندازه‌گیری نمایید. در گزارش خود توضیح دهید:

• آیا TTFT افزایش یافته است؟

• آیا Throughput کاهش یافته است؟

• آیا استفاده از GPU تغییر کرده است؟

• علت این تغییرات از دیدگاه معماری چیست؟

گام چهارم: استفاده از حافظه‌ی جانبی

در این مرحله LMCache را به گونه‌ای تنظیم کنید که داده‌های KV Cache بر روی یک NVMe SSD ذخیره شوند. سپس همان آزمایش‌های مراحل قبل را تکرار کرده و نتایج را با دو حالت قبلی مقایسه نمایید. به طور خاص، تاثیر انتقال داده بین:

- GPU HBM

- CPU DRAM

- NVMe SSD

را بر عملکرد سیستم تحلیل کنید.

گام پنجم: بررسی تاثیر Page Cache

در این مرحله تاثیر Page Cache سیستم عامل را بررسی نمایید. ابتدا سیستم را در حالت عادی اجرا کنید و سپس با فعال کردن گزینه‌ی

`use_odirect`

در LMCache از O_DIRECT استفاده کنید تا Page Cache دور زده شود. تاثیر این تغییر را بر معیارهای عملکرد تحلیل نمایید.

گام ششم: بررسی تاثیر اندازه‌ی بلاک

تمامی آزمایش‌های مراحل قبل را با اندازه‌های مختلف بلاک انجام دهید. به عنوان مثال اندازه‌های زیر را بررسی کنید:

2MB و 512KB، 128KB، 16KB، 4KB

مشخص کنید که تغییر اندازه‌ی بلاک چه تاثیری بر عملکرد سیستم و میزان انتقال داده دارد.

گام هفتم: تحلیل گلوگاه‌های معماری

در انتها، با استفاده از ابزارهایی مانند

- nvidia-smi

- Nsight Systems

- iostat

- fio

- perf

رفتار سیستم را تحلیل کرده و مشخص نمایید که در هر سناریوی آزمایش، گلوگاه اصلی عملکرد مربوط به کدام بخش از معماری کامپیوتر بوده است. به طور مشخص، هر آزمایش را در یکی از دسته‌های زیر طبقه‌بندی کنید:

- GPU Compute-Bound

- GPU Memory Capacity-Bound

- Memory Bandwidth-Bound

- PCIe Bandwidth-Bound

- Storage-Bound

- CPU-Bound

در نهایت، علت این رفتار را با استناد به ویژگی‌های معماری هر سطح از سلسله‌مراتب حافظه (ظرفیت، پهنای باند و تاخیر) و نمودارهای به‌دست‌آمده توضیح دهید.

معیارهای ارزیابی

در این پروژه هدف تنها اجرای موفق برنامه نیست، بلکه تحلیل رفتار معماری سیستم در سناریوهای مختلف نیز بخش مهمی از ارزیابی خواهد بود. دانشجویان باید برای تمامی آزمایش‌ها معیارهای زیر را اندازه‌گیری، ثبت و تحلیل کنند.

۱. Time to First Token (TTFT)

زمان موردنیاز از لحظه‌ی ارسال درخواست تا تولید اولین توکن توسط مدل.

۲. Throughput

تعداد توکن‌های تولیدشده در واحد زمان.

۳. GPU Utilization

میزان استفاده از واحدهای پردازشی کارت گرافیکی در هر سناریو.

۴. GPU Memory Usage

میزان مصرف حافظه‌ی کارت گرافیکی و تغییرات آن در هنگام KV Cache Offloading.

۵. CPU Utilization

درصد استفاده از پردازنده‌ی مرکزی و تاثیر انتقال بخشی از بار کاری به CPU.

۶. Main Memory Usage

میزان استفاده از حافظه‌ی اصلی سیستم و تغییرات آن در هنگام استفاده از LMCache.

۷. PCIe Activity

بررسی میزان انتقال داده بین CPU و GPU و تاثیر آن بر عملکرد سیستم.

۸. Storage Bandwidth

پهنای باند مورد استفاده برای دسترسی به حافظه‌ی جانبی در هنگام استفاده از NVMe SSD.

۹. Storage Latency

تاخیر دسترسی به حافظه‌ی جانبی و تاثیر آن بر عملکرد استنتاج.

۱۰. Page Cache Behavior

بررسی تاثیر استفاده یا عدم استفاده از Page Cache بر عملکرد سیستم.

در گزارش نهایی، دانشجویان باید علاوه بر ارائه‌ی نمودارهای مربوط به معیارهای فوق، مشخص کنند که در هر آزمایش، گلوگاه اصلی سیستم مربوط به کدام بخش از معماری کامپیوتر بوده است و علت آن را بر اساس ویژگی‌های معماری هر سطح از سلسله‌مراتب حافظه توضیح دهند.

قسمت امتیازی

دانشجویان می‌توانند برای کسب امتیاز اضافه، یک یا چند مورد از فعالیت‌های زیر را انجام دهند.

۱. بررسی تاثیر استفاده از RAM Disk به عنوان یک سطح میانی در سلسله‌مراتب حافظه و مقایسه‌ی آن با NVMe SSD.
۲. مقایسه‌ی عملکرد چند فایل سیستم مختلف (مانند EXT4، XFS، Btrfs و F2FS) بر روی حافظه‌ی NVMe و تحلیل دلیل تفاوت‌های مشاهده‌شده از دیدگاه معماری.
۳. بررسی تاثیر اندازه‌های مختلف KV Cache بر میزان انتقال داده، مصرف حافظه و عملکرد سیستم.
۴. بررسی تاثیر اندازه‌ی Batch Size بر معیارهای TTFT، Throughput و میزان استفاده از GPU.
۵. بررسی تاثیر استفاده از Pinned Memory در انتقال داده بین CPU و GPU و مقایسه‌ی آن با حافظه‌ی معمولی.
۶. رسم نمودار سلسله‌مراتب حافظه و مشخص کردن مسیر حرکت داده در هر یک از سناریوهای اجرا.
۷. تحلیل این موضوع که اگر گذرگاه PCIe با NVLink جایگزین شود، انتظار می‌رود کدام گلوگاه‌های سیستم کاهش پیدا کنند و علت آن چیست.

نحوه‌ی تحویل گزارش

تمامی فایل‌های پروژه باید در یک پوشه‌ی واحد تحویل داده شوند و شامل موارد زیر باشند.

۱. تمامی کدها، اسکریپت‌ها، فایل‌های پیکربندی و راهنمای اجرای پروژه.
۲. مشخصات کامل سخت‌افزار مورد استفاده شامل مدل GPU، پردازنده، حجم حافظه‌ی اصلی، نوع SSD و نسخه‌ی سیستم عامل.
۳. نتایج تمامی آزمایش‌ها به همراه فایل‌های خام خروجی ابزارهای اندازه‌گیری.
۴. نمودارهای مربوط به تمامی معیارهای ارزیابی شامل TTFT، Throughput، مصرف حافظه، استفاده از GPU، استفاده از CPU و سایر معیارهای اندازه‌گیری‌شده.
۵. تحلیل کامل نتایج و توضیح اینکه در هر آزمایش، گلوگاه اصلی سیستم مربوط به کدام بخش از معماری کامپیوتر بوده است.
۶. بحث و نتیجه‌گیری شامل مهم‌ترین یافته‌های پروژه، محدودیت‌های سیستم و پیشنهادهایی برای بهبود عملکرد.

لینک‌های مفید

- CUDA Toolkit Documentation
<https://docs.nvidia.com/cuda/>
- CUDA C++ Programming Guide
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- CUDA Samples
<https://github.com/NVIDIA/cuda-samples>
- NVIDIA Nsight Systems
<https://developer.nvidia.com/nsight-systems>

- NVIDIA Nsight Compute
<https://developer.nvidia.com/nsight-compute>
- NVIDIA System Management Interface (nvidia-smi)
<https://developer.nvidia.com/system-management-interface>
- CUDA Profiling Tools Interface (CUPTI)
<https://developer.nvidia.com/cupti>
- vLLM Documentation
<https://docs.vllm.ai/en/stable/>
- vLLM GitHub Repository
<https://github.com/vllm-project/vllm>
- LMCache Documentation
<https://docs.lmcache.ai/>
- LMCache Storage Backends
https://docs.lmcache.ai/kv_cache/storage_backends/local_storage.html
- GuideLLM Benchmark Tool
<https://github.com/vllm-project/guidellm>
- Flexible I/O Tester (fio)
<https://fio.readthedocs.io/>
- Linux perf Documentation
<https://perf.wiki.kernel.org/>
- NVIDIA Container Toolkit
<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>
- Installing NVIDIA Drivers on Ubuntu 24.04
<https://linuxconfig.org/how-to-install-nvidia-drivers-on-ubuntu-24-04>
- ArchWiki: EXT4
<https://wiki.archlinux.org/title/Ext4>
- ArchWiki: XFS
<https://wiki.archlinux.org/title/XFS>
- ArchWiki: Btrfs
<https://wiki.archlinux.org/title/Btrfs>
- ArchWiki: F2FS
<https://wiki.archlinux.org/title/F2FS>

پروژه پنجم: طراحی پردازنده چند هسته‌ای با پروتکل انسجام حافظه نهان MESI

مقدمه

در بخش‌های مختلف این درس با طراحی پردازنده‌های مبتنی بر معماری MIPS در قالب مدل‌های تک‌سیکلی، چندسیکلی و پایپ‌لاین آشنا شدید. در پردازنده‌های امروزی، افزایش کارایی تنها از طریق افزایش فرکانس امکان‌پذیر نیست و استفاده از چند هسته پردازشی به یکی از مهم‌ترین روش‌های افزایش توان محاسباتی تبدیل شده است. با این حال، وجود چند هسته که به صورت همزمان به حافظه دسترسی دارند، چالش جدیدی به نام *انسجام حافظه نهان*^{۴۱} را ایجاد می‌کند. در این پروژه هدف طراحی و پیاده‌سازی یک سامانه پردازشی دو هسته‌ای مبتنی بر معماری MIPS است. هر هسته دارای حافظه نهان داده اختصاصی بوده و ارتباط با حافظه اصلی از طریق یک گذرگاه مشترک انجام می‌شود. برای حفظ سازگاری داده‌ها میان حافظه‌های نهان، از پروتکل انسجام MESI استفاده خواهد شد.

اهداف

۱. آشنایی با معماری پردازنده‌های چند هسته‌ای.
۲. پیاده‌سازی یک پردازنده پایپ‌لاین MIPS و اتصال آن به حافظه نهان.
۳. درک و پیاده‌سازی پروتکل انسجام حافظه MESI.
۴. طراحی و پیاده‌سازی کنترلر گذرگاه مشترک.
۵. تحلیل تعامل میان هسته‌ها در زمان دسترسی همزمان به داده‌های مشترک.

مشخصات کلی سیستم

سیستم طراحی شده باید شامل دو هسته پردازشی مستقل مبتنی بر MIPS باشد که به صورت همزمان اجرا می‌شوند. هر هسته باید از مجموعه دستورات زیر پشتیبانی کند:

- دستورات نوع R: add, sub, and, or, slt
- دستورات نوع I: addi, ori, lw, sw
- دستورات کنترلی: beq, bne, j

هر هسته باید دارای یک حافظه نهان داده سطح اول (L1 Data Cache) مستقل باشد.

مشخصات پیشنهادی حافظه نهان

- ظرفیت کل: ۶۴ بایت
- اندازه هر بلاک: ۱۶ بایت
- تعداد خطوط: ۴
- نوع نگاشت: Direct-Mapped
- سیاست نوشتن: Write-Back
- ذخیره‌سازی بیت‌های وضعیت پروتکل MESI برای هر بلاک

⁴¹Cache Coherence

پروتکل انسجام MESI

برای هر بلاک موجود در حافظه نهان یکی از چهار وضعیت زیر باید نگهداری شود:

حالت	توضیح
M	داده تغییر کرده و با حافظه اصلی متفاوت است
E	داده تنها در یک Cache وجود دارد
S	داده میان چند Cache مشترک است
I	داده نامعتبر است

حداقل انتقال‌های زیر باید در طراحی پشتیبانی شوند:

- Read Miss
- Write Miss
- Shared → Modified
- Exclusive → Modified
- Invalidate
- Write Back

گام‌های پروژه

گام اول: طراحی هسته پردازشی
یک پردازنده MIPS پنج مرحله‌ای شامل مراحل زیر پیاده‌سازی کنید:

- IF
- ID
- EX
- MEM
- WB

پردازنده باید در هنگام وقوع Cache Miss امکان توقف (Stall) پایپ‌لاین را داشته باشد.

گام دوم: طراحی حافظه نهان

حافظه نهان داده با مشخصات ذکرشده را پیاده‌سازی نمایید. عملیات خواندن، نوشتن، جایگذاری بلاک و مدیریت وضعیت‌های MESI باید توسط این ماژول انجام شود.

گام سوم: طراحی کنترلر گذرگاه

یک Bus Arbiter طراحی کنید که وظایف زیر را برعهده داشته باشد:

- مدیریت دسترسی دو هسته به گذرگاه
- ارسال درخواست به حافظه اصلی

- ارسال سیگنال‌های Snoop

- اعمال عملیات Invalidate

در صورت ارسال همزمان درخواست توسط هر دو هسته، سیاست Round Robin برای تخصیص گذرگاه استفاده شود.

گام چهارم: طراحی برنامه آزمایشی

یک برنامه اسمبلی طراحی کنید که در آن هر دو هسته به مجموعه‌ای از آدرس‌های مشترک دسترسی داشته باشند و رفتار پروتکل MESI قابل مشاهده باشد.

سناریوهای آزمون

تست‌بنچ باید حداقل سناریوهای زیر را بررسی کند:

۱. خواندن یک داده مشترک توسط هر دو هسته.
 ۲. نوشتن یک هسته روی داده مشترک و نامعتبر شدن نسخه موجود در هسته دیگر.
 ۳. انتقال یک بلاک از حالت Modified به حافظه اصلی در زمان خروج از Cache.
- دانشجویان می‌توانند سناریوهای تکمیلی دیگری نیز طراحی کنند.

قسمت امتیازی: Cache Victim

به منظور کاهش تعداد Cache Miss‌های ناشی از تداخل، یک Victim Cache به معماری اضافه کنید. مشخصات این بخش:

- ظرفیت ۴ بلاک

- Fully Associative

- قرارگیری بین L1 Cache و Bus Arbiter

در صورت اخراج یک بلاک از حافظه نهان، بلاک ابتدا وارد Cache Victim شده و در صورت وقوع Miss در L1، پیش از مراجعه به گذرگاه، Cache Victim بررسی شود.

نحوه تحویل

تمامی موارد زیر باید در یک فایل فشرده تحویل داده شوند:

۱. تمامی فایل‌های Verilog

۲. تمامی فایل‌های Testbench

۳. برنامه اسمبلی مورد استفاده

۴. فایل راهنمای اجرا (README)

۵. گزارش پروژه شامل:

- معماری طراحی شده

- ساختار ماژول‌ها
- نحوه پیاده‌سازی MESI
- نتایج اجرای تست‌ها
- چالش‌ها و مشکلات پروژه

پروژه ششم: طراحی پردازنده با معماری دوحالت و راهاندازی امن

مقدمه

یکی از مهم‌ترین چالش‌های سامانه‌های امروزی، اطمینان از صحت نرم‌افزار پیش از اجرای آن است. در بسیاری از سامانه‌های تعبیه‌شده و اینترنت اشیا، پردازنده باید پیش از اجرای برنامه اصلی، از معتبر بودن آن اطمینان حاصل کند. به همین منظور، پردازنده‌های مدرن از مکانیزم‌هایی مانند سطوح دسترسی مختلف، حافظه‌های محافظت‌شده و فرآیند راهاندازی امن^{۴۲} استفاده می‌کنند.

در این پروژه هدف طراحی و پیاده‌سازی یک پردازنده مبتنی بر معماری MIPS است که از دو سطح دسترسی مستقل پشتیبانی می‌کند. این پردازنده در زمان روشن شدن سیستم، برنامه‌ای را از حافظه اصلی دریافت کرده، صحت آن را با استفاده از یک واحد سخت‌افزاری تولید هش بررسی می‌کند و تنها در صورت معتبر بودن، اجرای برنامه را آغاز می‌نماید. این پروژه با هدف آشنایی دانشجویان با مفاهیم سطوح دسترسی، طراحی واحدهای جانبی مبتنی بر Memory-Mapped I/O و توسعه مجموعه دستورات پردازنده تعریف شده است.

اهداف

۱. آشنایی با مفهوم سطوح دسترسی در پردازنده‌ها.
۲. توسعه معماری MIPS از طریق افزودن دستور جدید به ISA.
۳. طراحی و پیاده‌سازی واحدهای جانبی مبتنی بر MMIO.
۴. آشنایی با مکانیزم راهاندازی امن.
۵. طراحی یک شتاب‌دهنده سخت‌افزاری^{۴۳} ساده برای محاسبه هش.
۶. پیاده‌سازی کنترل دسترسی در سطح سخت‌افزار.

مشخصات کلی سیستم

پردازنده باید مبتنی بر معماری MIPS بوده و حداقل از مجموعه دستورات زیر پشتیبانی کند:

- دستورات نوع R: add, sub, and, or, xor, slt
- دستورات نوع I: addi, ori, xori, lw, sw
- دستورات کنترلی: beq, bne, j

علاوه بر دستورات فوق، باید یک دستور اختصاصی جدید با نام زیر به مجموعه دستورات اضافه شود:

DROP_PRIV

این دستور تنها در حالت امن قابل اجرا بوده و وظیفه آن انتقال پردازنده از حالت امن به حالت کاربر است.

^{۴۲}Secure Boot

^{۴۳}Hardware Accelerator

معماری دو حالت

پردازنده باید دارای یک ثبات داخلی یک‌بیتی به نام Mode_Bit باشد.

مقدار	حالت پردازنده
۱	Secure Mode
۰	User Mode

پس از روشن شدن سیستم، مقدار اولیه این ثبات باید برابر ۱ باشد و پردازنده در حالت Secure Mode قرار گیرد.

فضای آدرس‌دهی حافظه

برای جلوگیری از تفاوت زیاد میان طراحی‌ها، فضای حافظه سیستم باید مطابق جدول زیر تعریف شود:

بازه آدرس	کاربرد
0x0000 – 0x0FFF	Boot ROM
0x1000 – 0x1FFF	User RAM
0x2000 – 0x20FF	Crypto Accelerator Registers

محدودیت‌های دسترسی

در حالت Secure Mode:

- واکنشی دستور از Boot ROM مجاز است.
- دسترسی به RAM مجاز است.
- دسترسی به رجیسترهای واحد هش مجاز است.
- اجرای دستور DROP_PRIV مجاز است.

در حالت User Mode:

- واکنشی دستور از Boot ROM ممنوع است.
- دسترسی به رجیسترهای واحد هش ممنوع است.
- تنها دسترسی به فضای RAM مجاز است.

این محدودیت‌ها باید توسط سخت‌افزار اعمال شوند و نباید به نرم‌افزار وابسته باشند.

گام‌های پروژه

گام اول: توسعه پردازنده

پردازنده MIPS را توسعه داده و ثبات Mode_Bit و دستور DROP_PRIV را به آن اضافه کنید. این دستور باید تنها در حالت Secure قابل اجرا باشد.

گام دوم: طراحی Memory Router

ماژولی طراحی کنید که براساس آدرس مقصد و مقدار Mode_Bit، دسترسی‌های حافظه را کنترل نماید. این ماژول باید:

- درخواست‌های مجاز را عبور دهد.
- درخواست‌های غیرمجاز را مسدود کند.

گام سوم: طراحی شتاب‌دهنده هش

یک واحد سخت‌افزاری تولید هش طراحی کنید.
برای جلوگیری از پیچیدگی بیش از حد پروژه، استفاده از یکی از روش‌های زیر کافی است:

• CRC32

• XOR Accumulator

این واحد باید از طریق رجیسترهای MMIO کنترل شود.

گام چهارم: توسعه برنامه راه‌انداز

برنامه‌ای در Boot ROM قرار دهید که مراحل زیر را انجام دهد:

۱. خواندن برنامه کاربر از RAM
۲. ارسال داده‌ها به واحد هش
۳. دریافت مقدار نهایی هش
۴. مقایسه با مقدار مرجع
۵. اجرای DROP_PRIV در صورت معتبر بودن برنامه
۶. انتقال کنترل به برنامه کاربر

سناریوهای آزمون

تست‌بنچ باید حداقل دو سناریوی زیر را بررسی کند.
سناریوی اول: بوت موفق

۱. برنامه معتبر در RAM قرار می‌گیرد.
۲. مقدار هش با مقدار مرجع یکسان است.
۳. دستور DROP_PRIV اجرا می‌شود.
۴. پردازنده وارد User Mode می‌شود.
۵. برنامه کاربر اجرا می‌شود.

سناریوی دوم: برنامه دستکاری‌شده

۱. برنامه موجود در RAM تغییر داده می‌شود.
۲. مقدار هش با مقدار مرجع متفاوت است.
۳. پردازنده در حالت Secure باقی می‌ماند.
۴. برنامه کاربر اجرا نمی‌شود.

دانشجویان می‌توانند سناریوهای تکمیلی دیگری نیز طراحی و آزمایش نمایند.

قسمت امتیازی: Hardware Execution Watchdog

برای افزایش امنیت سیستم، یک ماژول مستقل طراحی کنید که به سیگنال‌های PC و Mode_Bit متصل باشد. این ماژول باید رفتار پردازنده را در حالت Secure Mode پایش کند. در صورتی که هنگام فعال بودن Secure Mode، شمارنده برنامه به صورت غیرمجاز وارد محدوده آدرس‌های RAM شود، ماژول باید بدون دخالت پردازنده، سیگنال Reset کل سیستم را فعال نماید. این بخش باید به صورت کاملاً سخت‌افزاری و مستقل از نرم‌افزار پیاده‌سازی شود.

نحوه تحویل

تمامی موارد زیر باید در یک فایل فشرده تحویل داده شوند:

۱. تمامی فایل‌های Verilog
۲. تمامی فایل‌های Testbench
۳. برنامه راه‌انداز موجود در ROM
۴. برنامه کاربر مورد استفاده در آزمایش‌ها
۵. فایل راهنمای اجرا (README)
۶. گزارش پروژه شامل:
 - معماری سیستم
 - نحوه پیاده‌سازی Mode_Bit
 - طراحی Router Memory
 - طراحی واحد هش
 - نتایج آزمایش‌ها
 - چالش‌ها و مشکلات پروژه

پروژه هفتم: طراحی پردازنده سوپراسکالر دو-مسیره (Dual-Issue Superscalar MIPS)

مقدمه

در طول ترم، شما موفق به طراحی یک پردازنده پایپ‌لاین ۵ مرحله‌ای MIPS شده‌اید که در هر چرخه ساعت (ایده‌آل) یک دستور را اجرا می‌کند ($IPC = 1$). اما برای افزایش بیشتر کارایی بدون افزایش فرکانس ساعت، پردازنده‌های مدرن از معماری‌های سوپراسکالر (Superscalar) بهره می‌برند. چالش اصلی در این پروژه، ارتقای پردازنده پیشین برای واکنشی، کدگشایی و اجرای دو دستورالعمل به صورت همزمان در یک چرخه ساعت است. هدف کلی این پروژه، درک عمیق توازی در سطح دستورالعمل (ILP) و مدیریت پیچیده مخاطرات (Hazards) در سخت‌افزار است. شما از مفاهیمی چون فایل‌های ثابتی چند-پورته، ALUهای موازی، و منطق توزیع پویای دستورات (Hardware Dispatcher) در محیط Logisim Evolution استفاده خواهید کرد.

اهداف

۱. ارتقای مسیر داده (Datapath) برای پشتیبانی از واکنشی و اجرای دو دستور همزمان.
۲. طراحی یک فایل ثابتی پیشرفته با ۴ پورت خواندن و ۲ پورت نوشتن.
۳. پیاده‌سازی واحد تشخیص مخاطره پیچیده (Hazard/Issue Unit) برای بررسی وابستگی‌های RAW, WAW, WAR بین دو دستورالعمل.
۴. انجام تحلیل کارایی (Performance Analysis) و محاسبه میزان بهبود سرعت (Speedup) کد فیوناچی.

مشخصات کلی سیستم

- معماری پایه: 32-bit MIPS Dual-Issue In-Order
- دستورات نوع R: add, sub, and, or, slt
- دستورات نوع I: lw, sw, addi, beq
- دستورات کنترلی: j
- NOP (No Operation): این دستور هیچ وضعیت مداری را تغییر نمی‌دهد و زمانی مجاز/لازم است که واحد توزیع‌کننده مجبور به متوقف کردن (Stall) یکی از مسیرهای اجرا به دلیل وابستگی داده شود. در این حالت به جای دستور دوم، یک NOP به خط لوله تزریق می‌شود.

ویژگی اصلی معماری: واحد توزیع دستورات (Instruction Dispatcher)

واحد توزیع‌کننده سخت‌افزاری (Dispatcher) در مرحله Decode قرار دارد و دارای یک سیگنال کنترلی ۱ بیتی به نام Issue_Mode است که وضعیت اجرای همزمان را تعیین می‌کند. مقدار اولیه آن پس از واکنشی هر جفت دستور محاسبه می‌شود.

- Dual Issue: هیچ وابستگی داده‌ای بین دستور ۱ (قدیمی‌تر) و دستور ۲ (جدیدتر) نیست. هر دو با هم به ALU می‌روند.
- Single Issue: مخاطره داده تشخیص داده شده. دستور ۱ صادر می‌شود و دستور ۲ در مرحله Decode متوقف (Stall) می‌ماند.

فضای آدرس دهی حافظه و ارتقای ساختار

با توجه به اینکه حافظه فعلی شما دارای آدرس دهی ۹ بیتی (۵۱۲ خانه) و به صورت کلمه‌ای (Word-Accessible) است، معماری سوپراسکالر نیازمند تغییرات زیر است:

- **حافظه دستورات (Instruction Memory):** برای واکشی دو دستور همزمان، باید از حافظه دو پورته (Dual-Port ROM/RAM) استفاده کنید. پورت اول آدرس PC و پورت دوم آدرس $PC + 1$ را می‌خواند.
- **حافظه داده (Data Memory):** می‌تواند همان ساختار تک پورته ۹ بیتی باقی بماند. اما این موضوع باعث ایجاد یک چالش جدید طراحی (مخاطره ساختاری) می‌شود که باید در واحد توزیع مدیریت شود.

قوانین صدور همزمان

در حالت Dual Issue (مقدار ۱):

- اجرای همزمان دستور ۱ و دستور ۲ مجاز است.
- نوشتن همزمان در دو ثبات مختلف در فایل ثباتی مجاز است.
- شرط: هیچ یک از ثبات‌های مبدا دستور ۲، نباید ثبات مقصد دستور ۱ باشد (No RAW Hazard).

در حالت Single Issue (مقدار ۰):

- ورود دستور ۲ به مرحله اجرا ممنوع است (تزریق NOP در مسیر دوم).
- PC تنها به اندازه ۴ بایت (یک دستور) به جلو می‌رود، نه ۸ بایت.
- دلیل: دستور ۲ برای اجرا به نتیجه دستور ۱ نیاز دارد یا محدودیت ساختاری (مانند دو دستور lw همزمان با حافظه تک-پورته) وجود دارد.
- تأکید می‌شود که اعمال این محدودیت‌ها (تشخیص وابستگی و استال کردن) باید کاملاً سخت‌افزاری و درون منطق ترکیبی Dispatcher باشد.

گام‌های پروژه

گام اول: توسعه هسته پردازنده (مسیر داده دوگانه) باید حافظه دستورات را طوری تغییر دهید که در هر چرخه، آدرس PC و $PC + 4$ را همزمان بخواند (خروجی ۶۴ بیتی). Register File باید بازطراحی شود تا دارای ۴ پورت خواندن (Read Ports) و ۲ پورت نوشتن (Write Ports) باشد. در مرحله اجرا (EX) باید دو ALU موازی قرار دهید (یکی برای Lane A و دیگری برای Lane B).

گام دوم: طراحی کنترل‌کننده توزیع (Hardware Dispatcher) مازولی در مرحله Decode طراحی کنید که عملوندهای دستور ۱ و دستور ۲ را با هم مقایسه کند. اگر دستور ۲ به نتیجه دستور ۱ وابسته بود، سیگنال $Issue_Mode = 0$ تولید شده، دستور ۲ متوقف شده و به جای آن یک حباب (Bubble/NOP) به ALU دوم فرستاده می‌شود.

گام سوم: طراحی تست‌بنچ و تحلیل کارایی (Performance Analysis) برنامه‌ای (مانند محاسبه دنباله فیبوناچی) بنویسید که قابلیت اجرای موازی بالایی داشته باشد (با بازنویسی کد اسمبلی برای کاهش وابستگی‌های متوالی).

۱. کد را روی پردازنده تک-مسیره قبلی خود اجرا کنید و تعداد چرخه‌ها را بشمارید.
۲. کد را روی پردازنده سوپراسکالر اجرا کنید و تعداد چرخه‌ها را بشمارید.
۳. نرخ IPC (دستورالعمل بر چرخه) و میزان Speedup را محاسبه و در گزارش قید کنید.

سناریوهای آزمون

سناریوی اول: اجرای موفق دو دستور مستقل

۱. شرایط اولیه: دستور اول add \$t0, \$t1, \$t2 و دستور دوم sub \$t3, \$t4, \$t5 واکنشی می‌شوند.
۲. عملیات مورد انتظار: واحد Dispatcher هیچ وابستگی RAW، WAW یا WAR تشخیص نمی‌دهد. Issue_Mode = 1 می‌شود.
۳. نتیجه نهایی: هر دو دستور در یک کلاک از ALU عبور کرده و PC هشت بایت به جلو می‌پرد ($PC = PC + 8$).

سناریوی دوم: تشخیص مخاطره داده (Data Hazard)

۱. شرایط اولیه: دستور اول add \$t0, \$t1, \$t2 و دستور دوم sub \$t3, \$t0, \$t4 واکنشی می‌شوند.
۲. عملیات مورد انتظار: Dispatcher تشخیص می‌دهد که دستور دوم ثابت \$t0 (مقصد دستور اول) را به عنوان مبدأ نیاز دارد. Issue_Mode = 0 می‌شود.
۳. نتیجه: دستور اول به Lane A می‌رود. به Lane B یک NOP فرستاده می‌شود. PC تنها ۴ بایت به جلو می‌رود تا در چرخه بعدی، دستور sub به عنوان دستور اول گروه جدید مجدداً بررسی شود.

سناریوی سوم: طراحی یک برنامه و مقایسه عملکرد آن تحت معماری سوپراسکالر

۱. شرایط اول: پیاده‌سازی یک برنامه با قابلیت موازی سازی بالا و استفاده از تمام منابع پردازنده جدید.
۲. نتیجه: رسیدن به speedup تقریباً برابر با ۲.

بخش امتیازی (اختیاری): اضافه کردن شمارنده‌های کارایی سخت‌افزاری (Performance Counters)

در پردازنده‌های واقعی، برای اندازه‌گیری دقیق کارایی (مانند IPC) از شمارنده‌های سخت‌افزاری درون تراشه‌ای استفاده می‌شود. برای کسب نمره امتیازی، سه ثابت شمارنده (Counter) مستقل و ۳۲-بیتی به هسته پردازنده خود اضافه کنید که موارد زیر را در طول اجرای برنامه فیونچی به صورت خودکار بشمارند:

۱. تعداد کل چرخه‌های ساعت (Total Clock Cycles): با هر لبه بالا رونده کلاک یکی اضافه شود.
 ۲. تعداد دستورات صادر شده (Total Issued Instructions): در صورت صدور تکی (مقدار ۱) و در صورت صدور جفتی (مقدار ۲) به آن افزوده شود.
 ۳. تعداد کل توقف‌ها (Total Stalls): هر بار که واحد Dispatcher به دلیل وابستگی داده، مسیر را متوقف کرده و حباب (NOP) تزریق می‌کند، یکی به این شمارنده اضافه شود.
- شما باید در گزارش خود، مقادیر نهایی این سخت‌افزار را پس از اتمام برنامه ثبت کرده و از آن‌ها برای اثبات نرخ IPC خود استفاده کنید.

موارد تحویلی

تمامی موارد زیر باید در یک فایل فشرده تحویل داده شوند:

۱. تمامی فایل‌های شماتیک Logisim Evolution (.circ) و ماژول‌های Verilog (در صورت استفاده از بلوک‌های HDL).

۲. تمامی فایل‌های Testbench.

۳. برنامه‌های اسمبلی (از جمله برنامه فیوناچی) اجرا شده بر روی پردازنده.

۴. فایل راهنمای اجرا (README).

۵. گزارش پروژه شامل موارد زیر:

- معماری سیستم و نمودار بلوکی (Block Diagram) مسیر داده دوگانه.
- نحوه طراحی و منطق بولی واحد توزیع دستورات (Dispatcher).
- طراحی حافظه و Register File مورد نیاز.
- تحلیل کارایی: نتایج آزمایش‌ها، محاسبه IPC، و مقایسه زمان اجرای برنامه فیوناچی بین حالت پایپ‌لاین معمولی و سوپراسکالر.

پروژه هشتم: طراحی واحد هدایت داده (Data Forwarding Unit)

مقدمه

در طول ترم، شما یک پردازنده پایپ‌لاین ۵ مرحله‌ای MIPS طراحی کردید. احتمالاً متوجه شده‌اید که در صورت وجود وابستگی داده (مانند زمانی که یک دستور به نتیجه دستور قبلی نیاز دارد)، خط لوله باید برای چندین چرخه متوقف (Stall) شود تا داده در فایل‌های ثبتی نوشته شده و مجدداً خوانده شود. این توقف‌ها کارایی (IPC) پردازنده را به شدت کاهش می‌دهند. این پروژه به حل این چالش می‌پردازد. هدف کلی این پروژه، طراحی و پیاده‌سازی یک واحد هدایت داده (Data Forwarding / Bypassing) است. شما با استفاده از مالتی‌پلکسرها و منطق ترکیبی پیشرفته در محیط Logisim Evolution، نتایج را مستقیماً از مراحل Memory یا WriteBack به ورودی‌های ALU بازمی‌گردانید تا از توقف خط لوله جلوگیری کنید. این پروژه یکی از مهم‌ترین مفاهیم در معماری پردازنده‌های مدرن را به شما آموزش می‌دهد.

اهداف

۱. درک عمیق مخاطرات داده (Data Hazards) از نوع Read-After-Write (RAW).
۲. طراحی مسیر داده پیچیده با افزودن مالتی‌پلکسرهای ۳-به-۱ در مرحله اجرا (Execute).
۳. پیاده‌سازی منطق ترکیبی مقایسه‌گرها برای واحد Forwarding Unit.
۴. پیاده‌سازی واحد تشخیص مخاطره (Hazard Detection Unit) برای مدیریت استثنائات (مانند Load-Use Hazard).
۵. تحلیل و مقایسه زمان اجرای یک برنامه (مانند فیوناچی) قبل و بعد از اضافه شدن این واحد.

مشخصات کلی سیستم

- معماری پایه: 32-bit MIPS 5-Stage Pipeline (Single-Issue)
- دستورات نوع R: add, sub, and, or, slt
- دستورات نوع I: lw, sw, addi, beq
- دستورات کنترلی: j

ویژگی اصلی معماری: سیگنال‌های کنترلی هدایت داده

هسته اصلی این پروژه واحد Forwarding Unit است که در مرحله EX قرار می‌گیرد. این واحد دو سیگنال کنترلی ۲ بیتی به نام‌های ForwardA (برای کنترل ورودی اول ALU) و ForwardB (برای کنترل ورودی دوم ALU) تولید می‌کند. مقادیر این سیگنال‌ها بر اساس مقایسه ثبات‌های مبدأ (Rs, Rt) و ثبات‌های مقصد (Rd) در مراحل بعدی خط لوله تعیین می‌شود.

منبع داده (Data Source)	ForwardA / ForwardB
(ID/EX) File Register : حالت عادی، داده مستقیماً از مرحله کدگذاری می‌آید.	00
(EX/MEM) حافظه : داده از خروجی ALU دستور قبلی هدایت می‌شود.	10
(MEM/WB) مرحله نوشتن : داده از خروجی حافظه یا ALU دستور قدیمی‌تر هدایت می‌شود.	01

محدودیت‌های دسترسی (قوانین هدایت و توقف)

پردازنده باید بر اساس وابستگی‌های داده به صورت خودکار بین حالت‌های زیر تصمیم‌گیری کند:
قوانین هدایت داده (Forwarding):

- **مخاطره EX:** اگر ثبات مقصد دستوری که در مرحله MEM است با ثبات مبدأ دستوری که در مرحله EX است برابر باشد، و سیگنال RegWrite آن فعال باشد، داده باید از EX/MEM هدایت شود (مقدار 10).
- **مخاطره MEM:** اگر ثبات مقصد در مرحله WB با مبدأ مرحله EX برابر باشد، داده باید از MEM/WB هدایت شود (مقدار 01).
- **اولویت‌بندی:** اگر هر دو مخاطره بالا همزمان رخ دهند (مانند نوشتن متوالی در یک ثبات)، مخاطره EX به دلیل جدیدتر بودن داده اولویت دارد.

محدودیت توقف (Load-Use Hazard):

- **شرط توقف:** اگر دستوری در مرحله EX یک دستور lw باشد و ثبات مقصد آن با یکی از ثبات‌های مبدأ دستور در مرحله ID برابر باشد، Forwarding به تنهایی کافی نیست.
- **اقدام سخت‌افزاری:** واحد Hazard Detection باید خط لوله را برای یک چرخه متوقف کند ($PCWrite = 0$) و به جای دستور موجود در ID، یک حباب (NOP) به مرحله EX ارسال کند ($IF/IDWrite = 0$).

گام‌های پروژه

گام اول: اصلاح مسیر داده در مرحله اجرا (Execute) در مقابل هر دو ورودی ALU خود، یک مالتی‌پلکسر ۳-به-۱ اضافه کنید. ورودی 00 همان سیم قبلی از فایل ثباتی است. ورودی 10 را از خروجی ثبات خط لوله EX/MEM و ورودی 01 را از خروجی ثبات خط لوله MEM/WB (پس از مالتی‌پلکسر حافظه) با استفاده از Tunnel‌های Logisim متصل کنید.

گام دوم: طراحی منطق Forwarding Unit یک زیرمدار مستقل طراحی کنید که شماره ثبات‌های R_s و R_t از مرحله EX، ثبات R_d از مرحله MEM، و ثبات R_d از مرحله WB را دریافت کند. با استفاده از مقایسه‌گرها (Comparators) و گیت‌های منطقی، سیگنال‌های ForwardA و ForwardB را تولید کنید. بررسی $R_d \neq \$0$ الزامی است.

گام سوم: طراحی Hazard Detection Unit در مرحله Decode، ماژولی طراحی کنید که وابستگی‌های Load-Use را بررسی کند. این ماژول در صورت تشخیص این مخاطره، باید جلوی آپدیت شدن PC و IF/ID را بگیرد و سیگنال‌های کنترلی مرحله EX را صفر کند (Flush/Bubble).

گام چهارم: ارزیابی نرم‌افزاری و محک‌زنی (Benchmarking) برنامه فیوناچی را به زبان اسمبلی نوشته و به کد ماشین تبدیل کنید. این برنامه دارای وابستگی‌های داده متوالی است. یک بار پردازنده را بدون Forwarding Unit اجرا کنید (با استفاده از NOP‌های دستی در کد) و بار دیگر کد بدون NOP را روی پردازنده نهایی اجرا کرده و تفاوت چرخه‌های ساعت را ثبت کنید.

سناریوهای آزمون

سناریوی اول: اجرای موفق با هدایت داده مستقیم (EX Hazard)

۱. شرایط اولیه: دستور $add \$t0, \$t1, \$t2$ وارد مرحله MEM شده و دستور $sub \$t3, \$t0, \$t4$ در مرحله EX قرار دارد.

۲. عملیات مورد انتظار: واحد Forwarding Unit برابری R_s دستور دوم با R_d دستور اول را تشخیص داده و $ForwardA = 10$ را تنظیم می‌کند.

۳. نتیجه نهایی: بدون هیچ‌گونه توقف (Stall)، نتیجه جمع مستقیماً وارد ورودی تفریق‌کننده می‌شود.

سناریوی دوم: تشخیص مخاطره بارگذاری-استفاده (Load-Use Hazard)

۱. شرایط اولیه: دستور lw \$t0, 0(\$t1) در مرحله EX و دستور add \$t2, \$t0, \$t3 در مرحله ID قرار دارد.
۲. عملیات مورد انتظار: واحد Hazard Detection Unit وابستگی را قبل از ورود به ALU تشخیص می‌دهد. خط لوله متوقف می‌شود.
۳. نتیجه (تأخیر کنترل شده): در چرخه بعدی، دستور add در مرحله ID باقی می‌ماند، یک NOP وارد مرحله EX می‌شود. در چرخه بعدتر، داده از MEM/WB با استفاده از Forwarding Unit (01) به ALU هدایت می‌شود و دستور add اجرا می‌گردد.

بخش امتیازی (اختیاری)

در معماری پایه، دستورات پرش (beq) در مرحله EX بررسی می‌شوند که به معنای ۳ چرخه جریمه در صورت پرش است. بخش امتیازی شامل انتقال مقایسه‌گر پرش از مرحله EX به مرحله ID است. این کار نیازمند طراحی یک شبکه Forwarding اختصاصی و کوچک در مرحله Decode است تا در صورتی که ثبات‌های شرط پرش به تازگی محاسبه شده‌اند، داده‌ها مستقیماً به مقایسه‌گر پرش در مرحله ID هدایت شوند.

موارد تحویلی

تمامی موارد زیر باید در یک فایل فشرده تحویل داده شوند:

۱. تمامی فایل‌های پروژه در Logisim Evolution شامل زیرمدارهای جداگانه برای Forwarding Unit و Hazard Detection.
۲. فایل کدهای اسمبلی و معادل کد ماشین (محتویات ROM).
۳. فایل راهنمای اجرا (README).
۴. گزارش پروژه شامل موارد زیر:
 - نقشه مسیر داده ارتقا یافته به همراه مسیرهای بازخورد (Bypass Paths).
 - جداول درستی (Truth Tables) یا معادلات منطقی استفاده شده در واحد هدایت داده.
 - نتایج آزمایش‌ها و سناریوها.
 - تحلیل کارایی: مقایسه دقیق تعداد چرخه‌های ساعت مورد نیاز برای محاسبه فیبوناچی دهم در پردازنده معمولی (با توقف) نسبت به پردازنده مجهز به شبکه هدایت داده.